

# YOLO 目标检测算法

魏舟

2026 年 2 月 7 日

## Contents

# 1 目标检测基础理论

## 1.1 目标检测任务定义

目标检测的核心目标是“在图像中找到感兴趣的目标，并确定其类别和位置”。

- **分类 (Classification)**: 识别图像中的物体属于哪个类别（如：猫、狗、汽车）。
- **定位 (Localization)**: 确定物体在图像中的具体空间位置。
- **边界框 (Bounding Box)**: 通常由四元组表示：
  - 中心点坐标格式:  $(x_{center}, y_{center}, w, h)$
  - 左上角坐标格式:  $(x_{min}, y_{min}, x_{max}, y_{max})$
- **置信度 (Confidence Score)**: 模型对其预测结果真实性的信心程度，计算公式通常为  $P(Object) \times \text{IoU}_{pred}^{truth}$ 。

## 1.2 评价指标 (Metrics)

为了客观衡量模型性能，通常使用以下指标：

### 1.2.1 交并比 (IoU)

交并比用于衡量预测框  $B_p$  与真实框  $B_g$  的重叠程度：

$$\text{IoU} = \frac{\text{Area}(B_p \cap B_g)}{\text{Area}(B_p \cup B_g)}$$

### 1.2.2 非极大值抑制 (NMS)

在预测阶段，模型会对同一个物体生成多个重叠的候选框。**NMS** 的作用是：

1. 按照置信度得分对所有候选框进行排序。
2. 保留得分最高的框，并删除与之 IoU 大于预设阈值的所有其他框。
3. 重复此过程直到所有框被处理。

### 1.2.3 mAP (mean Average Precision)

这是衡量检测器综合性能的最重要指标：

- **Precision (精确率)**: 查准率，所有预测为正样本中实际为正的比率。
- **Recall (召回率)**: 查全率，所有实际正样本中被预测出来的比率。
- **AP**: P-R 曲线下的面积。
- **mAP**: 对所有类别的 AP 取算术平均。通常关注 mAP@0.5 或 mAP@.5:.95。

## 2 YOLO 核心思想：One-Stage Detector

### 2.1 设计哲学

YOLO (You Only Look Once) 的核心理念是将目标检测重新定义为一个单一的回归问题 (Regression Problem)。

- **统一网络**：从输入图像到边界框坐标和类别的概率，直接通过一个卷积神经网络完成推理。
- **网格划分 (Grid Cell)**：
  - 将输入图像划分为  $S \times S$  个网格。
  - 如果物体的中心落在某个网格内，该网格就负责检测该物体。
  - 每个网格预测  $B$  个边界框及这些框的置信度，以及  $C$  个类别的概率。
- **全局信息**：由于使用全图信息进行训练和推理，YOLO 相比滑动窗口方法，背景误检 (False Positives) 更少。

### 2.2 损失函数结构 (Loss Function)

YOLOv1 使用均方误差 (MSE) 作为损失函数。为了平衡坐标定位、目标置信度和类别分类，损失函数由四部分加权组成：

$$\begin{aligned} L = & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} \left[ (x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{obj} (C_i - \hat{C}_i)^2 \\ & + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{I}_{ij}^{noobj} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{I}_i^{obj} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))^2 \end{aligned} \quad (1)$$

#### 2.2.1 关键点解析

- **权重系数**： $\lambda_{coord}$  (通常取 5) 用于增加坐标损失的权重； $\lambda_{noobj}$  (通常取 0.5) 用于惩罚不包含物体的背景框，防止负样本淹没正样本。
- **平方根处理**：对宽  $w$  和高  $h$  取平方根，是为了减小大尺度边界框对误差的敏感度 (使小框的偏移比大框的相同像素偏移产生更大的 Loss)。
- **指示函数**： $\mathbb{I}_{ij}^{obj}$  表示第  $i$  个网格的第  $j$  个预测框负责该物体。

## 3 YOLO 经典版本演进

### 3.1 YOLOv1: 开创先河

YOLOv1 证明了单阶段检测器在实时性上的巨大潜力。

- **网络架构 (Backbone):** 参考了 GoogLeNet, 包含 24 个卷积层和 2 个全连接层, 输入分辨率为  $448 \times 448$ 。
- **局限性分析:**
  - **空间约束:** 每个网格只能预测一个类别, 且只有两个边界框, 导致其对密集目标 (如: 鸟群) 检测效果较差。
  - **泛化能力:** 对物体的长宽比和尺度变化较为敏感。
  - **定位误差:** 主要的误差来源是定位不准 (Localization Error)。

### 3.2 YOLOv2 (YOLO9000): 更好、更快、更强

v2 版本引入了多项改进, 使其在保持速度的同时大幅提升了精度。

- **Batch Normalization:** 每一层卷积后都加入 BN 层, 收敛更快, 并提升了 2% 的 mAP。
- **Anchor Boxes 的先验知识:**
  - 放弃了全连接层预测坐标, 改用 Anchor Boxes。
  - **Dimension Clusters:** 使用 K-means 聚类算法在训练集中自动找到最合适的 Anchor 宽高比。
- **Pass-through 层:** 将高分辨率特征图 (浅层) 与低分辨率特征图 (深层) 拼接, 增强了对小目标的检测能力。
- **多尺度训练:** 每隔 10 个 batch 随机改变输入图像的尺寸 (从 320 到 608), 增强模型的尺度适应性。

### 3.3 YOLOv3: 工业界的里程碑

v3 是目前学术界和工业界参考最多的经典版本, 结构非常稳健。

- **Darknet-53 特征提取器:** 引入了残差连接 (Residual Connections), 比 Darknet-19 更深, 精度接近 ResNet-101 但速度更快。
- **多尺度预测 (FPN 思想):**
  - 在三个不同的尺度上进行预测 ( $13 \times 13$ ,  $26 \times 26$ ,  $52 \times 52$ )。
  - 借鉴了特征金字塔网络 (FPN) 的特征融合方式。
- **逻辑回归替代 Softmax:**
  - 使用多个独立逻辑分类器 (Binary Cross Entropy) 替代全类别的 Softmax。
  - 支持了目标类别的层级关系 (如: 同一个物体既是 “女人” 又是 “人”)。

## 4 现代 YOLO 优化策略 (v4 - v11+)

### 4.1 Backbone 增强

主干网络的改进旨在以更低的计算成本提取更具表达力的特征。

- **CSPNet (Cross Stage Partial Networks):**
  - 将特征图分为两部分，一部分经过密集块 (Dense Block)，另一部分直接与处理后的特征图合并。
  - 解决了梯度信息重复的问题，在减少计算量的同时保持甚至提升了精度。
- **RepVGG 与结构重参数化:**
  - **训练时:** 使用多分支结构 ( $3 \times 3$ ,  $1 \times 1$  及直连) 来获取更好的特征表达。
  - **推理时:** 将所有分支融合为单一的  $3 \times 3$  卷积层，实现“推理无损加速”。这是 YOLOv6 及以后版本的核心。

### 4.2 Neck 层结构

Neck 层负责特征融合，连接 Backbone 与 Prediction Head。

- **PANet (Path Aggregation Network):**
  - 在 FPN 的基础上增加了一个自底向上的路径 (Bottom-up Path)，使底层定位信息更容易传递到高层特征。
- **SPP/SPPF (Spatial Pyramid Pooling - Fast):**
  - 通过不同维度的最大池化并拼接，极大地增加了感受野。
  - **SPPF** 是 YOLOv5 提出的优化版本，将串行池化代替并行池化，速度大幅提升。

### 4.3 数据增强与训练技巧 (Bag of Freebies)

这些技巧只增加训练成本，而不影响推理时间。

- **Mosaic 数据增强:**
  - 将四张图片拼接为一张，丰富了检测目标的上下文，变相增加了 Batch Size。
- **Label Smoothing:**
  - 防止模型对分类过于自信，提升泛化能力。
- **IoU Loss 的演进:**
  - **GIoU:** 解决了 IoU 为 0 时梯度消失的问题。
  - **DIoU:** 考虑了边界框中心点的距离。
  - **CIoU:** 同时考虑了重叠面积、中心点距离和长宽比的对称性。

#### 4.3.1 最新前沿 (v10/v11)

- **NMS-Free:** 通过双重分配 (Dual Assignment) 消除对 NMS 的依赖 (v10 核心)。
- **C3k2/C2f 结构:** 针对不同尺度进行更精细的特征管理。

## 5 实战与部署总结

### 5.1 开发环境配置

目前 YOLO 系列 (尤其是 v5, v8, v10, v11) 主要基于 Ultralytics 框架, 其生态极大地简化了开发流程。

- **框架选择:**
  - `pip install ultralytics`: 一键安装环境, 集成训练、验证、预测。
  - **硬件依赖:** CUDA 工具包与 cuDNN 的版本匹配。
- **数据集准备:**
  - **YOLO 格式:** 每张图对应一个 .txt 文件, 内容为 `<class-id> <x> <y> <w> <h>` (归一化坐标)。
  - **格式转换:**
    - \* **Pascal VOC:** XML 标签格式。
    - \* **MS COCO:** JSON 嵌套格式。
  - 使用 `data.yaml` 定义数据集路径和类别名称。

### 5.2 模型量化与加速

为了满足嵌入式设备或高并发服务器的实时性要求, 通常需要对模型进行转换和压缩。

#### 5.2.1 ONNX 导出注意事项

ONNX 是模型部署的中间媒介。导出时需注意:

- **动态维度 (Dynamic Axes):** 是否允许输入图片尺寸可变。
- **算子兼容性:** 部分 YOLO 的自定义层 (如某些版本的 Upsample) 在导出时可能需要调整 `opset_version`。
- **后处理集成:** 是否将 NMS 逻辑包含在 ONNX 模型内部。

### 5.2.2 TensorRT 部署流程

TensorRT 是 NVIDIA 推出的高性能推理引擎，其核心在于对模型进行重构。

- **层融合 (Layer Fusion):** 将卷积、偏移和激活函数融合为一个核函数 (Kernel)。
- **量化 (Quantization):**
  - **FP16:** 在精度几乎无损的情况下，推理速度显著提升。
  - **INT8 (PTQ/QAT):** 通过校准集 (Calibration set) 将权重压缩至 8 位，极大地减少内存占用，但需注意精度回退风险。
- **推理引擎生成:** 在目标设备上生成专用的 `.engine` 文件，以发挥最大算力。

Table 1: 常见部署格式对比

| 格式                 | 平台         | 优势      | 适用场景           |
|--------------------|------------|---------|----------------|
| PyTorch (.pt)      | GPU        | 灵活、易于调试 | 实验、训练阶段        |
| ONNX (.onnx)       | 跨平台        | 通用性强    | 中间转换、CPU 推理    |
| TensorRT (.engine) | NVIDIA GPU | 极致性能    | 边缘计算、实时视频流     |
| TFLite (.tflite)   | 移动端/端侧     | 轻量化     | Android/iOS 应用 |

## 6 结语与展望

随着深度学习技术的飞速发展，YOLO 已经从最初的纯卷积架构演变为一个高度集成的视觉感知框架。

- **端到端检测 (NMS-free):** YOLOv10 引入的双重分配机制 (Dual Assignment) 使得模型不再依赖后处理 NMS，显著降低了推理延迟。
- **多模态与开放词汇 (OVD):** 未来的 YOLO 将不仅限于检测预定义的 80 个类别，通过结合 CLIP 等大模型，实现“用自然语言描述什么就检测什么”。
- **实时实例分割:** 如 YOLOv8/v11-seg，通过增加计算轻量级的 Proto 模块，使实时像素级理解成为可能。

## 7 参考文献与资源

### 7.1 核心论文列表

- **YOLOv1:** Redmon, et al. "You Only Look Once: Unified, Real-Time Object Detection." CVPR 2016.
- **YOLOv3:** Redmon, et al. "YOLOv3: An Incremental Improvement." arXiv 2018.

- **YOLOv4**: Bochkovskiy, et al. "YOLOv4: Optimal Speed and Accuracy of Object Detection." arXiv 2020.
- **YOLOv7**: Wang, et al. "YOLOv7: Trainable Bag-of-freebies Sets New State-of-the-art for Real-time Object Detectors." CVPR 2023.
- **YOLOv10**: Wang, et al. "YOLOv10: Real-Time End-to-End Object Detection." arXiv 2024.

## 7.2 优秀开源库与工具

- **Ultralytics (YOLOv8/v11)**: <https://github.com/ultralytics/ultralytics> ——目前最易用的生产力工具。
- **MMDetection**: <https://github.com/open-mmlab/mmdetection> ——覆盖几乎所有主流检测算法的学术级框架。
- **Roboflow**: 用于在线标注数据集并导出 YOLO 格式的利器。

## 7.3 学习建议

“不仅要看模型跑出的 mAP 分数，更要深入阅读其 `head.py` 和 `loss.py` 的源码。理解样本匹配（Assigner）的逻辑，比理解网络架构本身更重要。”